

lkzietbmf

April 6, 2025

[1]:

```
[7]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc

# Load dataset
data = pd.read_csv("C:/Users/SUDIP/Downloads/pima-indians-diabetes.data.csv")
print(data.head())

# Separate features and target
X = data.iloc[:, :-1] # Features
y = data.iloc[:, -1]  # Target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Initialize models
models = {
    'SVM': SVC(probability=True, random_state=42),
    'Naive Bayes': GaussianNB(),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
```

```

        'KNN': KNeighborsClassifier()
    }

    # Store metrics
    metrics = {}

    # ROC setup
    plt.figure(figsize=(10, 8))

    for name, model in models.items():
        # Train
        model.fit(X_train, y_train)

        # Predict
        y_pred = model.predict(X_test)

        # Metrics
        report = classification_report(y_test, y_pred, output_dict=True)
        metrics[name] = {
            'Accuracy': report['accuracy'],
            'Precision': report['weighted avg']['precision'],
            'Recall': report['weighted avg']['recall'],
            'F1-Score': report['weighted avg']['f1-score']
        }

        # Confusion Matrix
        cm = confusion_matrix(y_test, y_pred)
        plt.figure(figsize=(6,4))
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
        plt.title(f'Confusion Matrix - {name}')
        plt.xlabel('Predicted')
        plt.ylabel('Actual')
        plt.show()

        # ROC Curve
        y_prob = model.predict_proba(X_test)[:,-1]
        fpr, tpr, _ = roc_curve(y_test, y_prob)
        roc_auc = auc(fpr, tpr)
        plt.plot(fpr, tpr, label=f'{name} (AUC = {roc_auc:.2f})')

    # Final ROC curve
    plt.plot([0, 1], [0, 1], 'k--') # Diagonal line
    plt.title('ROC Curve Comparison')
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.legend(loc='lower right')
    plt.grid()

```

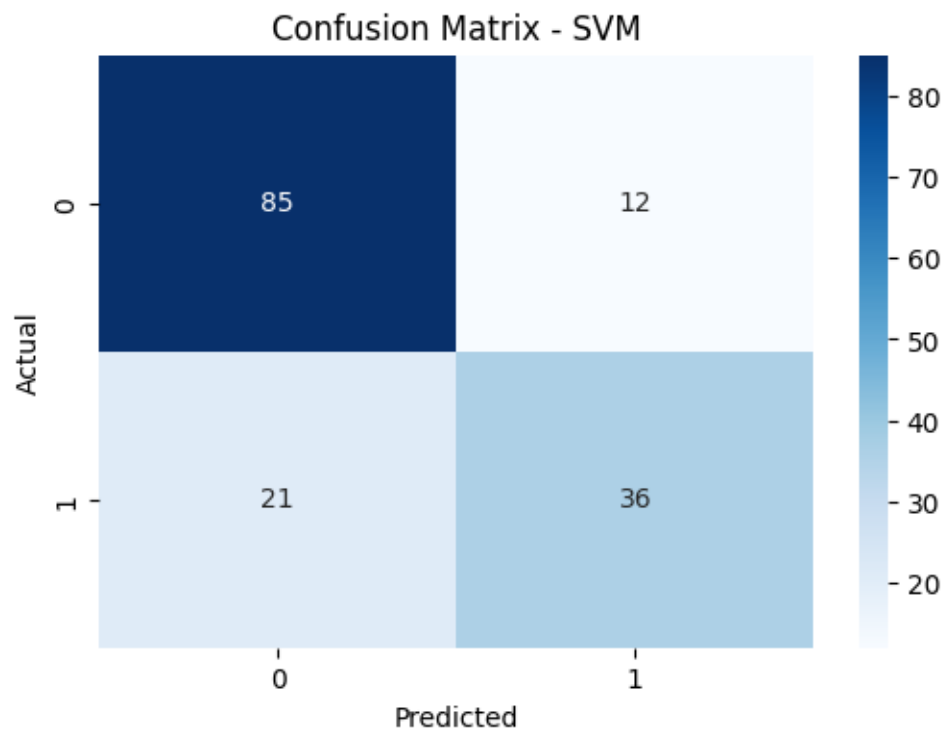
```
plt.show()

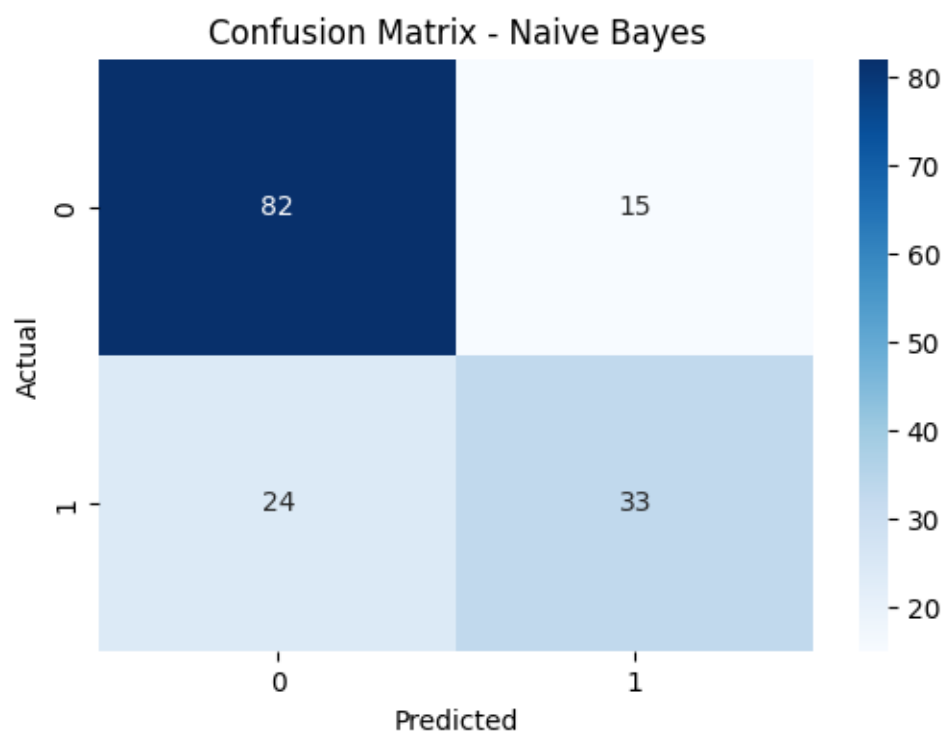
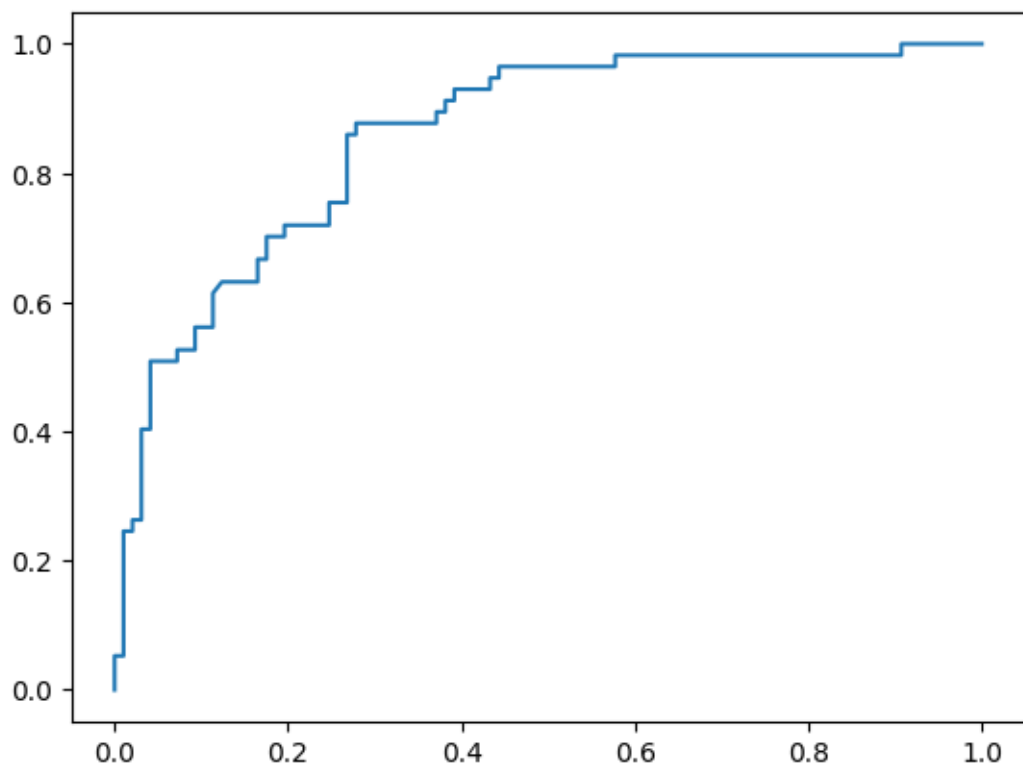
# Display metrics
metrics_df = pd.DataFrame(metrics).T
print("Performance Comparison:\n")
print(metrics_df)
```

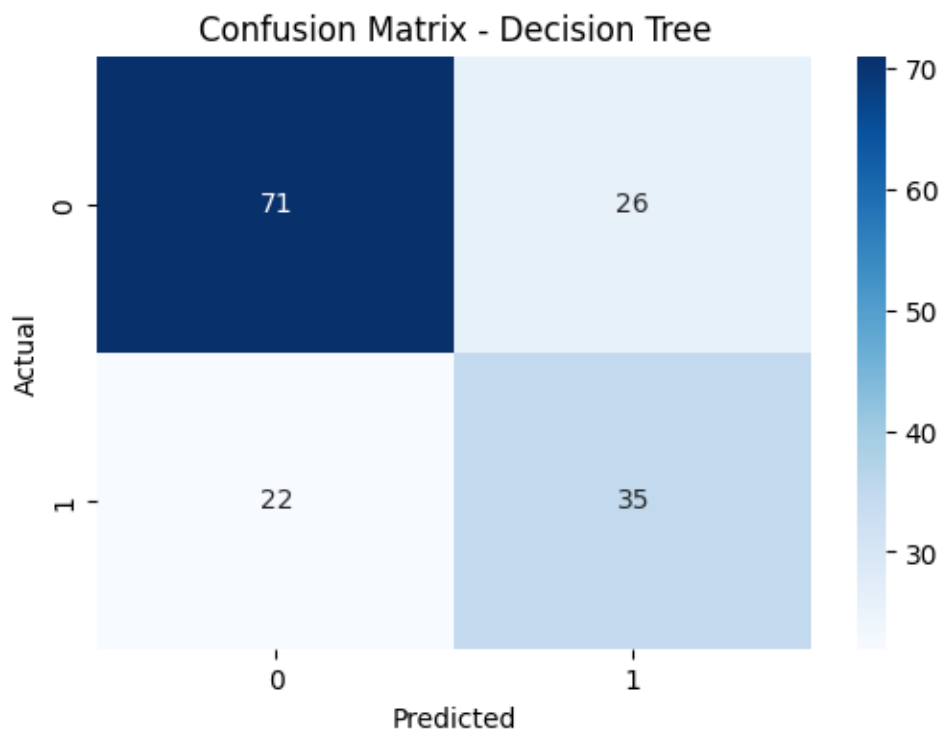
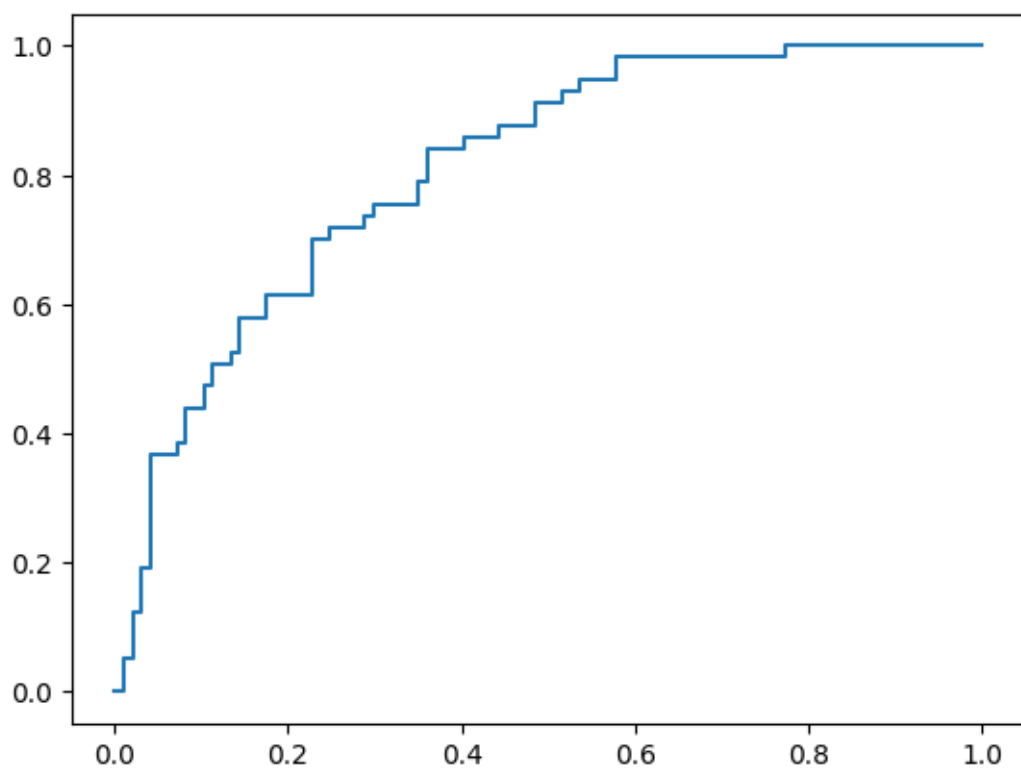
```

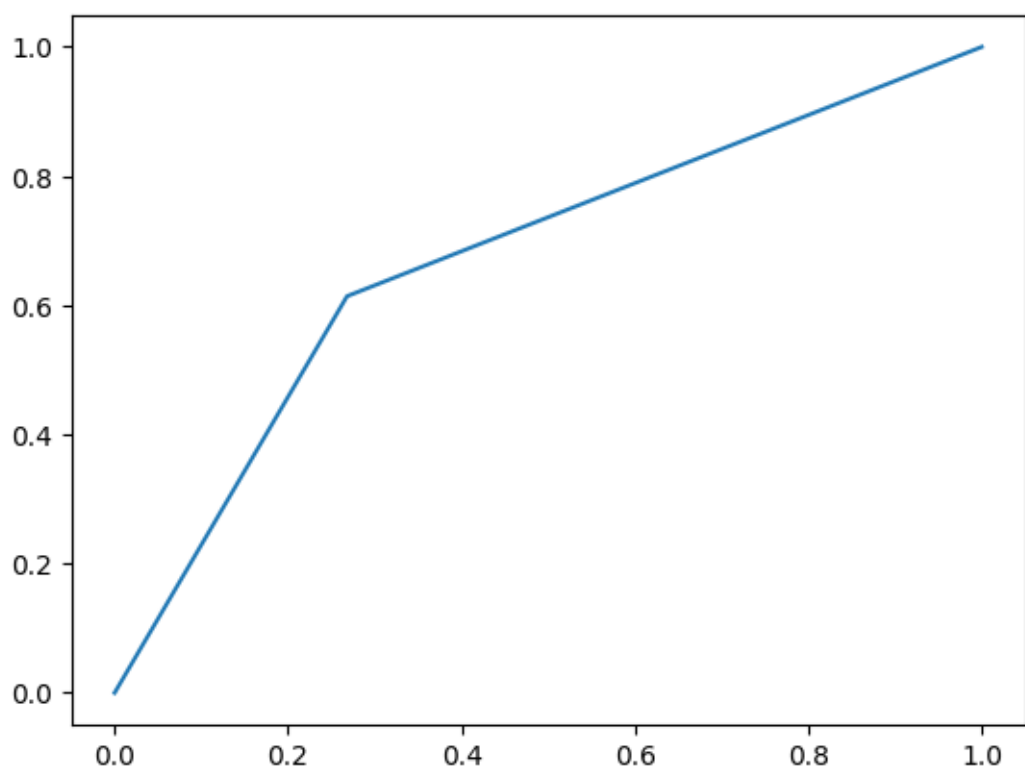
      6  148  72  35   0  33.6  0.627  50  1
0  1    85  66  29   0  26.6  0.351  31  0
1  8   183  64   0   0  23.3  0.672  32  1
2  1    89  66  23  94  28.1  0.167  21  0
3  0   137  40  35 168  43.1  2.288  33  1
4  5   116  74   0   0  25.6  0.201  30  0
```

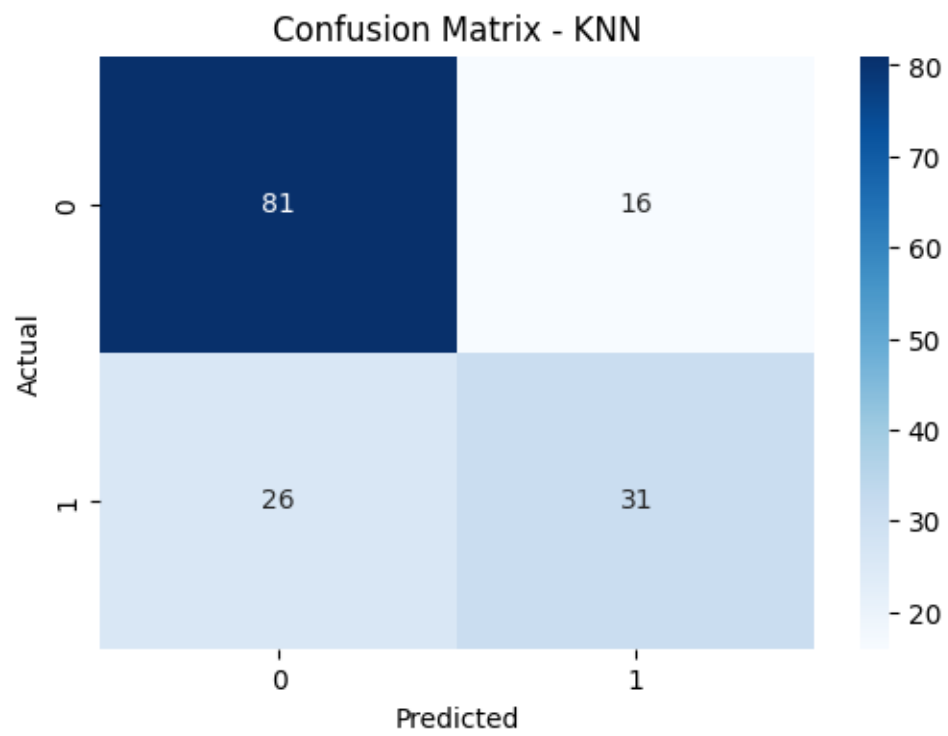
<Figure size 1000x800 with 0 Axes>

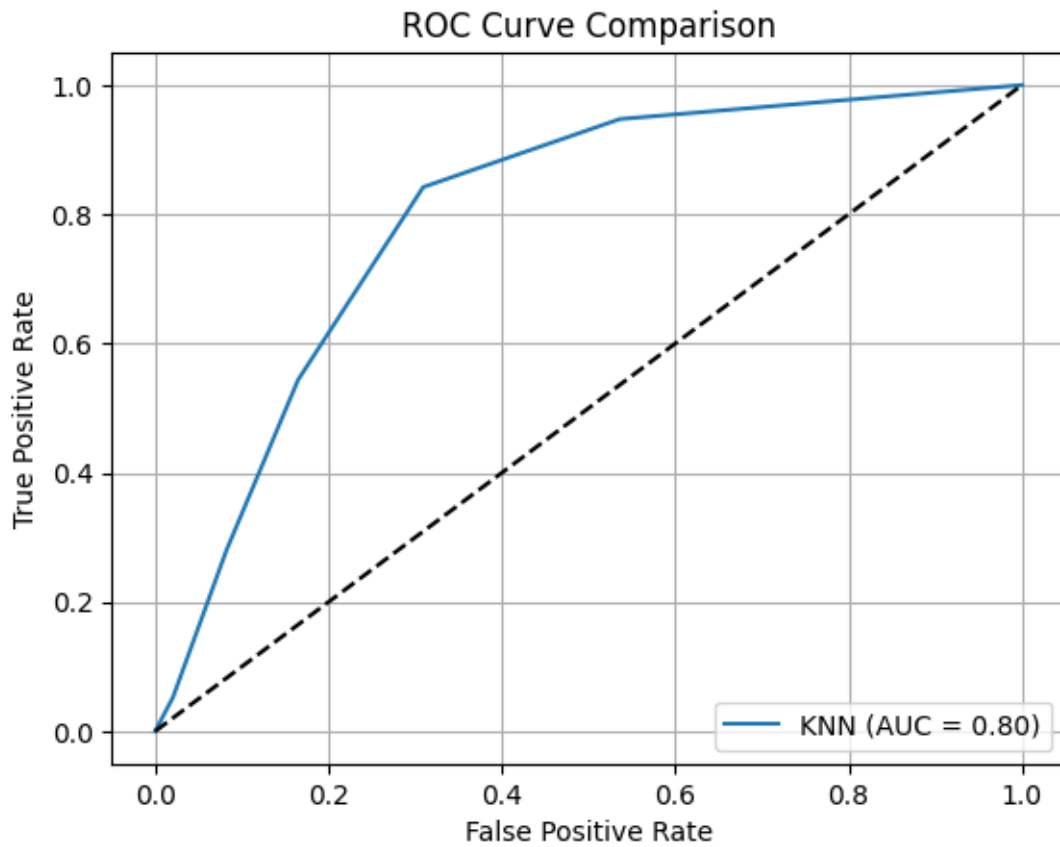












Performance Comparison:

	Accuracy	Precision	Recall	F1-Score
SVM	0.785714	0.782682	0.785714	0.781281
Naive Bayes	0.746753	0.741722	0.746753	0.741514
Decision Tree	0.688312	0.693238	0.688312	0.690314
KNN	0.727273	0.720946	0.727273	0.720845

[]: